Memoria de Práctica: Creación y consumo de una API

Miguel Angel Bolonio Zamora

1. Introducción y objetivos

El objetivo de este proyecto es diseñar, desarrollar y consumir una API utilizando Node.js y Express (con Router para modularizar las rutas). La finalidad principal ha sido construir un sistema modular que implemente un flujo de seguridad (autenticación y autorización), para después consumir estos servicios desde un cliente frontend simulado.

2. Tecnologías y paquetes utilizados

- **Entorno:** Node.js.
- **Framework:** Express.js (para la creación del servidor y enrutamiento).
- **Seguridad y Criptografía:**
  - **bcrypt:** Para el *hashing* seguro de las contraseñas.
  - **jsonwebtoken (JWT):** Para la generación de tokens de sesión sin estado.
  - **cookie-parser:** Para la gestión y validación de cookies HTTP.
  - **dotenv:** Para ocultar credenciales de la API.
  - **axios:** Para el manejo de peticiones HTTP.
- **Almacenamiento:** Simulación de base de datos en memoria mediante módulos exportables de JavaScript.
- **Pruebas:** Thunder Client (extensión VSCode).

3. Arquitectura del servidor y enrutamiento modular

Para hacer la API escalable en vez de concentrar todas las rutas en el archivo index.js, he implementado la herramienta express.Router para separar las responsabilidades por dominios y niveles de seguridad:

1. **Rutas públicas (public_routes.js):** Gestionan el registro y el login de usuarios. No requieren autenticación previa (/login, /registro).
2. **Rutas privadas de usuario (users_routes.js):** Muestran información básica del usuario. Requieren estar autenticado a través de un token jwt válido (/perfil).
3. **Rutas de administración (admin_routes.js):** Muestran información crítica. Requieren estar autenticado y tener el rol específico de "admin" (/panelControl).

En el archivo principal (index.js), estos módulos se conectan utilizando la función app.use(), actuando como el punto de entrada central y configurando los traductores necesarios (express.json() y express.urlencoded()) para interpretar y realizar una lectura correcta de las peticiones del frontend.

4. Flujo de Seguridad: Autenticación y Autorización

He diseñado la seguridad de la API dividiéndose en tres fases clave:

- **Fase 1: Registro.** Cuando un usuario se registra, la contraseña plana nunca es almacenada. Utilizo el algoritmo bcrypt de forma asíncrona (async/await) porque requiere un tiempo hacer esta operación y no bloquear el hilo de ejecución y guardo únicamente el hash generado junto al correo y el rol.
- **Fase 2: Login y generación del token.** Al iniciar sesión, se verifica la existencia de un usuario con el correo introducido y se compara su contraseña con el hash. Si coincide, se crea un JSON Web Token (JWT) que contiene el correo y el rol del usuario en un payload que he llamado infoToken. Para la persistencia de la sesión de cara al consumo desde el frontend, este token se envía al cliente inyectado en una cookie.
- **Fase 3: Middlewares Protectores.** He desarrollado dos middlewares personalizados para actuar en las peticiones a rutas protegidas:

    - verificarUsuario: Lee la cookie mediante cookie-parser y utiliza jwt.verify para comprobar la firma criptográfica. Si es válido, inyecta los datos del usuario en la petición (req.usuario = datosToken).
    - verificarRol: Intercepta la petición después de verificar al usuario y comprueba si req.usuario.rol es igual a 'admin', bloqueando el acceso con un error HTTP 403 (Forbidden) en caso negativo.

5. Dificultades y soluciones aplicadas

Durante el desarrollo de la API, surgieron algunos problemas y las soluciones que encontré fueron las siguientes:

- **Pérdida de datos en POST:** Inicialmente, las peticiones enviadas desde formularios HTML estáticos llegaban vacías al backend. Lo solucioné añadiendo el middleware global express.urlencoded({ extended: true }) para que Node pudiera traducir el formato tradicional de los formularios además del JSON.
- **Gestión de la asincronía en criptografía:** Se produjeron errores al intentar guardar contraseñas que aún se estaban procesando. Hice las rutas como funciones asíncronas, aplicando await a las funciones de bcrypt.hash y bcrypt.compare para pausar la ejecución hasta obtener el resultado de las mismas.
- **Control del flujo HTTP:** En la creación de los middlewares de seguridad, optimicé el control de flujo utilizando sentencias return res.status() para detener inmediatamente la ejecución en caso de error (como la falta de token) y evitar que Express intentara enviar múltiples respuestas al cliente.

6. Consumo de API externa desde el Backend (GitHub):

La alternativa al paquete deprecado request que he utilizado es el paquete axios, por su manejo nativo de promesas, permitiendo usar de forma simple la sintaxis async/await y hacer las peticiones asíncronas. El servidor creado en Node actúa como intermediario en lugar de que el frontend consulte directamente a GitHub, es mi propia API la que procesa la petición:

- **Rutas implementadas**: He creado los endpoints /usuario/githubUserInfo y /usuario/githubUserRepos.
- **Autenticación en la API de GitHub:** He consumido la API oficial de GitHub. La cual requiere autenticación para evitar bloqueos por limite de peticiones, mi servidor inyecta directamente en la cabecera HTTP de la petición el token de acceso personal suministrado por GitHub (headers: {'Authorization' : 'Bearer <token>').
- **Volcado de datos:** Mi servidor de Node captura el JSON devuelto por GitHub, lo imprime por consola y lo redirige simultáneamente al frontend con un código 200 para su renderizado.

Al intentar realizar la subida del código a mi repositorio personal del curso, el sistema de seguridad de GitHub me impidió hacer el commit al detectar que el token de la API estaba expuesto en el código en texto plano, para resolver esta vulnerabilidad he instalado el paquete dotenv el cual permite ocultar credenciales y información crítica de la aplicación, lo he inicializado en mi index.js para permitir la carga de variables de entorno y he guardado el token de GitHub en un archivo local oculto llamado .env, en el código el token se invoca de forma segura a través de process.env.token.

7. Consumo de la API desde el Frontend:

Para el desarrollo de la parte visual he creado vistas HTML interactivas (login.html, registro.html, y perfil.html) que consumen mi API de forma asíncrona usando JavaScript sin necesidad de recargar la página completa.

- **Peticiones con fetch:** En el archivo perfil.js, utilizo la función nativa fetch() para llamar al endpoint /usuario/githubUserInfo.
- **Gestión de la sesión:** Gracias a la configuración previa de las cookies con el flag httpOnly en el backend, el navegador web adjunta automáticamente el JWT en las peticiones. Esto permite que el middleware verificarUsuario valide la sesión sin que yo tenga que manipular el token manualmente en el lado del cliente.
- **Manipulación del DOM y redirecciones:** Si el servidor devuelve un error de autorización (ej. 401 Unauthorized), el script lo detecta y redirige automáticamente al usuario al login mediante window.location.href. Si la petición es exitosa, extraigo los datos JSON recibidos (la información combinada por Node y GitHub) y los inyecto en la interfaz modificando el innerHTML de los contenedores correspondientes para construir dinámicamente la tarjeta de perfil.